BÀI THỰC HÀNH CHƯƠNG 7

I. Sử dụng và điều chỉnh giao diện template login của app Admin

Ta có thể thêm chức năng đăng nhập (login) bằng cách thêm ánh xạ URL cho tính năng xác thực và sao chép template từ app **Admin** vào dự án **Bookr**. Nhưng trang login thường sẽ không đáp ứng hoàn toàn nhu cầu do có giao diện không thống nhất với giao diện các trang web thuộc dự án **Bookr**. Bài thực hành này sẽ điều chỉnh lại giao diện của trang login của app **Admin** để phù hợp với dự án **Bookr**.

1. Tạo một thư mục **templates/registration** trong thư mục của dự án **Bookr**.
2. Trang template login của app **Admin** được đặt trong thư mục **django/contrib/admin/templates/admin/login.html**. Các khối thẻ chính trong tập tin này có dạng như sau:

```
{% extends "admin/base_site.html" %}
{% load i18n static %}

{% block extrastyle %}{{ block.super }}…
{% endblock %}

{% block bodyclass %}{{ block.super }} login{% endblock %}
{% block usertools %}{% endblock %}
{% block nav-global %}{% endblock %}
{% block content_title %}{% endblock %}
{% block breadcrumbs %}{% endblock %}
```

   Sao chép tập tin login.html này vào thư mục **templates/registration** mới tạo ở **Bước 1.** Sau đó thực hiện các điều chỉnh trong tập tin này theo các bước tiếp sau đây.

3. Thay thế nội dung của thẻ **extends** bằng dòng mã sau để trang **login** sử dụng chung cấu trúc giao diện với các tính năng khác của dự án **Bookr**:

```
{% extends "base.html" %}
```

4. Phần lớn các dòng mã còn lại là không cần thiết, ta chỉ cần giữ lại thẻ **load** và nội dung của block **content**, phần giữ lại sẽ như sau:

```
{% load i18n static %}
{% block content %}
…
{% endblock %}
```

5. Thay thế dòng mã của thẻ form sau:

```
<form action="{{ app_path }}" method="post" id="login-form">
```

   Bằng dòng mã:

```
<form action="{% url 'accounts:login' %}" method="post"
id="login-form">
```

   Lý do là đường dẫn **app_path** không còn cần thiết và được thay bằng một URL cụ thể cho trang **login** của chính dự án **Bookr**.

   Tương tự, ta thay thế dòng mã chứa liên kết đến trang đổi mật khẩu:

```
{% url 'admin_password_reset' as password_reset_url %}
```

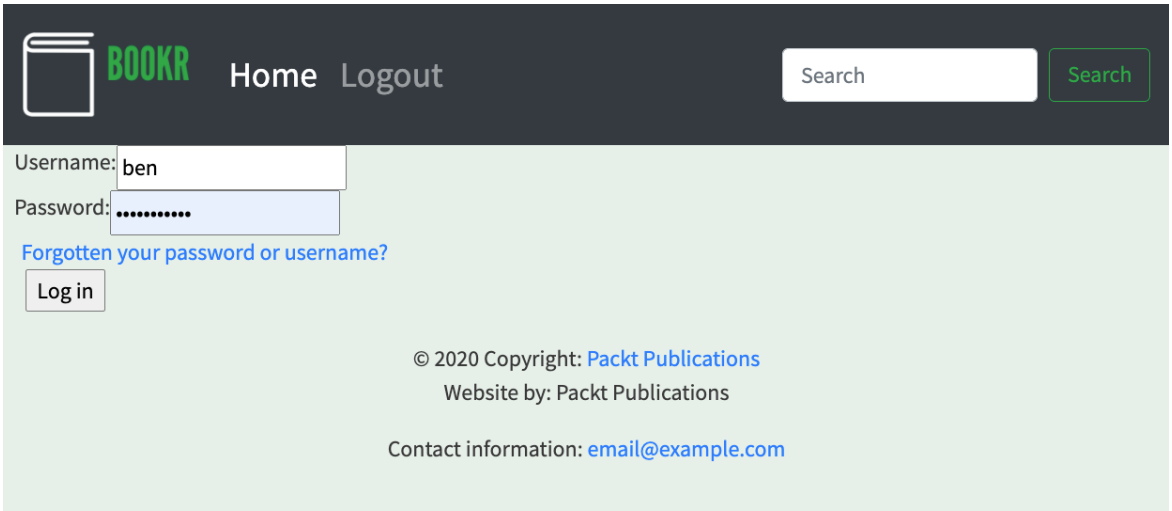
Bằng dòng mã sau:

```
{% url 'accounts:password_reset' as password_reset_url %}
```

6. Để sử dụng các view xác thực của Django, ta cần thêm ánh xạ URL cho chúng. Mở tập tin **urls.py** trong thư mục dự án **Bookr**, thực hiện **import** các gói **admin** và **auth** thuộc gói **django.contrib** và thêm 3 ánh xạ URL như sau:

```
from django.contrib import admin, auth
…
urlpatterns = [
    …
    path(
        "accounts/",
        include(
            ("django.contrib.auth.urls", "auth"),
            namespace="accounts"
        )
    ),
    path(
        "accounts/password_reset/done/",
        auth.views.PasswordResetDoneView.as_view(),
        name="password_reset_done",
    ),
    path(
        "accounts/reset/done/",
        auth.views.PasswordResetCompleteView.as_view(),
        name="password_reset_complete",
    ),
    …
]
```

7. Truy cập vào trang login theo URL http://127.0.0.1:8000/accounts/login/ kết quả sẽ như Hình I-1.



Hình I-1. Trang đăng nhập (login) của dự án Bookr

II. Thêm trang hồ sơ người dùng

Trong bài thực hành này, ta sẽ thêm một trang hồ sơ người dùng vào dự án **Bookr**. Để thêm trang này, ta cần thêm ánh xạ URL cũng như thêm view và template của trang. Trang hồ sơ sẽ hiển thị thông tin của người dùng với các thuộc tính tương ứng lấy từ đối tượng **user**: **username, first_name, last_name, date_joined, email, last_login**.

1. Thêm tập tin **bookr/views.py** với định nghĩa view **profile** như sau:

```
from django.shortcuts import render

def profile(request):
    return render(request, 'profile.html')
```

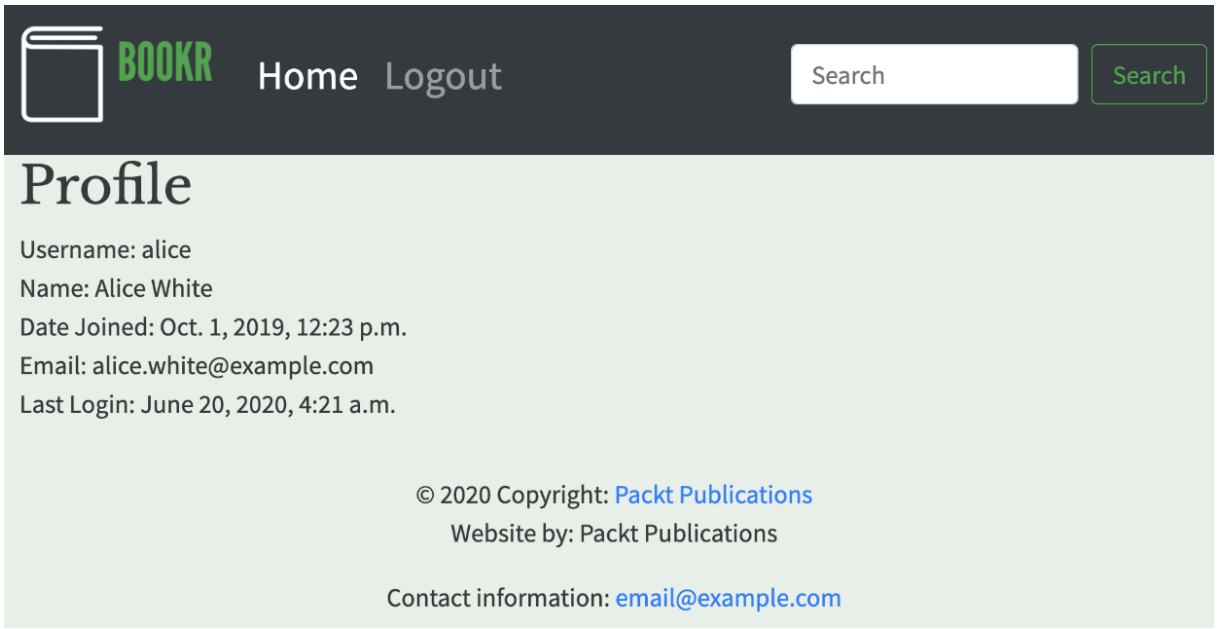
2. Trong thư mục **templates** của dự án **Bookr**, tạo mới một tập tin tên là **profile.html**. Trong tập tin này, ta khai báo kế thừa từ template **base.html** và định dạng giao diện hiển thị thông tin người dùng với các thuộc tính lấy từ đối tượng **request.user**. Đoạn mã nguồn của tập tin có thể ở dạng như sau:

```
{% extends "base.html" %}

{% block title %}Bookr{% endblock %}

{% block content %}
<h2>Profile</h2>
<div>
    <p>
        Username: {{ request.user.username }} <br>
        Name: {{ request.user.first_name }}
{{request.user.last_name }}<br>
        Date Joined: {{ request.user.date_joined }} <br>
        Email: {{ request.user.email }}<br>
        Last Login: {{ request.user.last_login }}<br>
    </p>
</div>
{% endblock %}
```

3. Cuối cùng, khai báo ánh xạ URL đến trang hồ sơ người dùng bằng cách thêm đoạn mã để **import** view **profile** và thêm đường dẫn (path) phần khai báo biến **urlpatterns** trong tập tin **books/urls.py**.

```
from bookr.views import profile

urlpatterns = [
    …
    path('accounts/profile/', profile, name='profile'),
    …
]
```

4. Sau khi đăng nhập ứng dụng bằng một tài khoản (ví dụ Alice) và mở trang hồ sơ người dùng bằng URL http://localhost:8000/accounts/profile/, giao diện của trang này sẽ như **Hình II-1**.



Hình II-1. Giao diện trang Hồ sơ người dùng (profile)

III. Thêm các decorator phục vụ xác thực vào view

Bài thực hành này sẽ sử dụng các decorator của app **Authentication** trong app **Review**. Ta cần đảm bảo rằng *chỉ những người dùng* mới có thể *chỉnh sửa các review* và *chỉ có người dùng là nhân viên* mới được *chỉnh sửa các nhà xuất bản (publisher).* Có vải cách khác nhau để làm việc này nên ta sẽ thử một vài phương án. Tất cả các chỉnh sửa sẽ thực hiện trong tập tin **reviews/views.py**.

1. Cách thứ 1: Sử dụng decorator để yêu cầu người dùng phải có quyền ***edit_publisher*** mới được sử dụng tính năng **publisher_edit**. Ta cần import và sử dụng decorator **permission_required**:

```
from django.contrib.auth.decorators import permission_required
…

@permission_required('edit_publisher')
def publisher_edit(request, pk=None):
    …
```

2. Cách thứ 2: Để kiểm tra theo cách phức tạp nhưng linh hoạt hơn, ta sử dụng decorator **user_passess_test** với tham số là một hàm (ví dụ tên là **is_staff_user()**) kiểm tra người dùng có phải là nhân viên không. Ta có thể tạm bỏ 2 dòng mã đã thêm vào ở **Bước 1** và thay bằng đoạn mã sau:

```
from django.contrib.auth.decorators import user_passes_test
…

def is_staff_user(user):
    return user.is_staff

@user_passes_test(is_staff_user)
def publisher_edit(request, pk=None):
    …
```

3. Để yêu cầu người dùng đã đăng nhập mới có thể chỉnh sửa review và đặt sách, ta sử dụng decorator **login_required** trước các view tương ứng như sau:

```
from django.contrib.auth.decorators import (login_required,
user_passes_test)
…

@login_required
def review_edit(request, book_pk, review_pk=None):
    …

@login_required
def book_media(request, pk):
    …
```

4. Trong phương thức **review_edit**, thêm đoạn mã để kiểm tra, yêu cầu người dùng phải là nhân viên hoặc là người soạn ra review mới được sửa. Ràng buộc này chỉ áp dụng trong trường hợp đang chỉnh sửa một review đã tồn tại Vì vậy, vị trí để thêm đoạn mã nằm sau

dòng mã truy xuất thành công đối tượng **Review**. Nếu người dùng không phải là nhân viên hoặc là người người tạo review thì ta cần xuất ra lỗi **PermissionDenied**:

```
…
from django.core.exceptions import PermissionDenied
from PIL import Image
from django.contrib import messages
…

@login_required
def review_edit(request, book_pk, review_pk=None):
    book = get_object_or_404(Book, pk=book_pk)

    if review_pk is not None:
        review = get_object_or_404(Review, book_id=book_pk,
pk=review_pk)
        user = request.user
        if not user.is_staff and review.creator.id != user.id:
            raise PermissionDenied
    else:
        review = None
…
```

Khi đó, nếu người dùng không phải là nhân viên mà cố chỉnh sửa một review của người dùng khác thì sẽ xuất hiện một thông báo lỗi **Forbidden** như trong **Hình III-1**.

Hình III-1. Truy cập để chỉnh sửa review bị cấm đối với người dùng không phải là nhân viên

IV. Bật tắt các link Login và Logout trong template base.html

Trong template **base.html** đã có sẵn một chỗ dành cho link **Logout** nằm trên phần tiêu đề của trang web. Đoạn mã đó như sau:

```
<li class="nav-item">
  <a class="nav-link" href="#">Logout</a>
</li>
```

Ta không muốn đường link **Logout** tiếp tục xuất hiện khi người dùng đã đăng xuất. Vì thế, trong bài thực hành này sẽ áp dụng điều kiện logic vào template để hiển thị link là **Login** hoặc **Logout** tùy trường hợp *người dùng đã được xác thực hay chưa*.

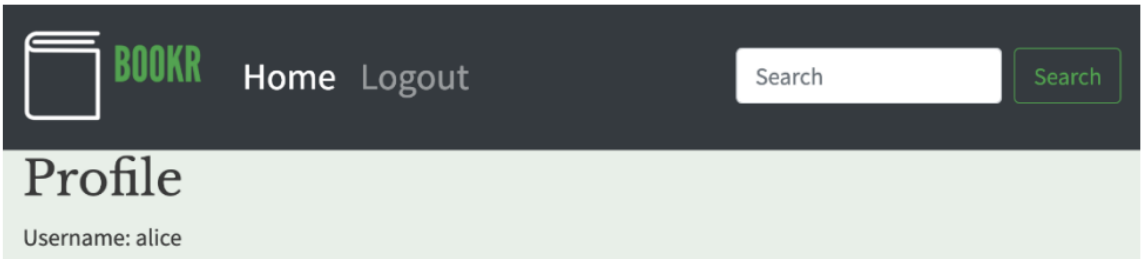
1. Chỉnh sửa lại tập tin **templates/base.html**. Sao chép cấu trúc của thẻ **<li>** của link **Logout** và tạo một thẻ **<li>** thứ hai cho link **Login**, link này sẽ mở trang **/accounts/login**. Sau khi thêm, đoạn mã của phần này sẽ như sau:

```
<li class="nav-item">
  <a class="nav-link" href="/accounts/logout">Logout</a>
</li>
<li class="nav-item">
  <a class="nav-link" href="/accounts/login">Login</a>
</li>
```

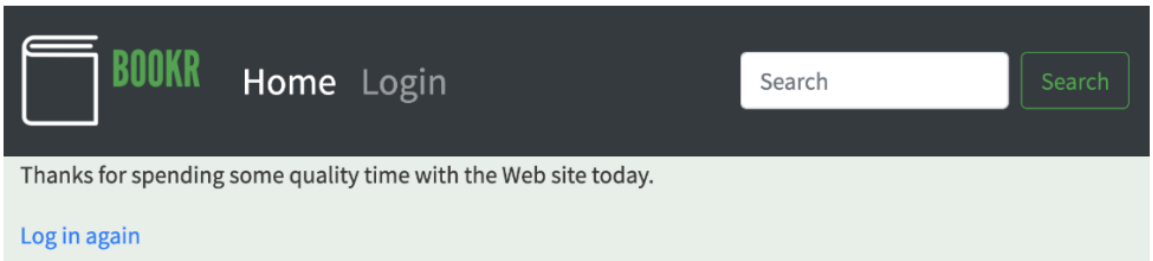
2. Đặt 2 thẻ **<li>** vào trong khối điều kiện **if … else … endif**. Điều kiện if là **user.is_authenticated** (người dùng đã xác thực/ đăng nhập hay chưa).

```
{% if user.is_authenticated %}
  <li class="nav-item">
     <a class="nav-link" href="/accounts/logout">Logout</a>
  </li>
{% else %}
  <li class="nav-item">
    <a class="nav-link" href="/accounts/login">Login</a>
  </li>
{% endif %}
```

3. Truy cập đến trang http://localhost:8000/accounts/profile/. Nếu người dùng đã đăng nhập thì giao diện sẽ như **Hình IV-1**. Nếu người dùng chưa đăng nhập hoặc sau khi đăng xuất (bấm vào nút Logout) thì giao diện sẽ như **Hình IV-2**.



Hình IV-1. Link Logout xuất hiện khi người dùng đã đăng nhập



Hình IV-2. Link Login xuất hiện khi người dùng chưa đăng nhập hoặc sau khi đăng xuất

V. Kiểm tra session key (Không bắt buộc)

Bài thực hành này sẽ truy vấn đến bảng session của CSDL **SQLite** để làm quen với dữ liệu về phiên được lưu trong CSDL. Sau đó, ta sẽ tạo một lệnh quản lý của Django để kiểm tra dữ liệu phiên đã lưu sử dụng **JSONSerializer**.

Sinh viên thực hành theo nội dung của phần **Exercise 9.05 – examining the session key** tại trang **457- 461** của **Giáo trình**.

VI. Lưu trữ các cuốn sách vừa xem vào đối tượng session

Mục đích của bài thực hành này là sử dụng đối tượng **session** để giữ lại thông tin của 10 cuốn sách gần nhất mà người dùng (đã xác thực/ đăng nhập) đã truy cập. Thông tin này sẽ được hiển thị trong trang hò sơ người dụng của dự án **Bookr**. Khi người dùng duyệt đến một cuốn sách, view **book_detail** sẽ được gọi. Ta sẽ chỉnh sửa lại logic của phương thức **book_detail** trong tập tin **reviews/views/py**. Trước tiên ta sẽ thêm một khóa vào đối tượng **session** có tên là **viewed_books**. Sau đó ta sẽ xuất ra các cuốn sách người dùng mới đọc trong thời gian gần nhất trong trang **Hồ sơ người dùng (profile).**

1. Chỉnh sửa phương thức **book_detail** trong tập tin **reviews/views/py**. Ta chỉ quan tâm đến việc thêm thông tin cho đối tượng **session** của người dùng **đã đăng nhập** nên cần thêm một điều kiện để kiểm tra người dùng đã xác thực/ đăng nhập hay chưa và đặt giá trị cho biến **max_viewed_books_length** (số cuốn sách tối đa mà người dùng đã xem trong thời gian gần đây nhất được lưu lại) là **10**.

```
def book_detail(request, pk):
    …
    if request.user.is_authenticated:
        max_viewed_books_length = 10
```

2. Trong khối điều kiện **if** này, thêm tiếp đoạn mã để truy xuất giá trị của biến phiên **request.session['viewed_books']**. Nếu giá trị này không tồn tại thì khởi tạo một **danh sách rỗng** ([])

```
        viewed_books = request.session.get( 'viewed_books', [])
```

3. Nếu khóa chính của cuốn sách hiện tại đã tồn tại (trong biến **viewed_books**), đoạn mã sau sẽ xóa bỏ khóa này:

```
        viewed_book = [book.id, book.title]
        if viewed_book in viewed_books:
            viewed_books.pop(viewed_books.index(viewed_book))
```

4. Tiếp tục thêm đoạn mã sau để thêm khóa chính của cuốn sách hiện tại vào đầu danh sách **viewed_books**:

```
        viewed_books.insert(0, viewed_book)
```

5. Thêm dòng mã sau để đảm bảo chỉ giữ lại **10** *cuốn sách đầu tiên* trong danh sách:

```
        viewed_books = viewed_books[ :max_viewed_books_length]
```

6. Tiếp tục, thêm dòng mã sau để gán lại giá trị của **session['viewed_books']** là **viewed_books** giá trị này sẽ xuất hiện trong các **request** về sau:

```
        request.session['viewed_books'] = viewed_books
```

7. Tương tự các view khác, ở phần cuối của phần định nghĩa hàm **book_detail**, gọi hàm **render()** cho template **reviews/book_detail.html:**

```
    return render(request, "reviews/book_detail.html", context)
```

Sau khi hoàn thành các bước trên, view **book_detail** sẽ có khối lệnh điều kiện sau:

```
def book_detail(request, pk):
    …
    if request.user.is_authenticated:
        max_viewed_books_length = 10
```

```
        viewed_books = request.session.get("viewed_books", [])
        viewed_book = [book.id, book.title]
        if viewed_book in viewed_books:
            viewed_books.pop(viewed_books.index(viewed_book))
        viewed_books.insert(0, viewed_book) viewed_books =
        viewed_books[:max_viewed_books_length]
        request.session["viewed_books"] = viewed_books return
    render(request, "reviews/book_detail.html", context)
```

8. Điều chỉnh lại bố cục và CSS của tập tin template **templates/profile.html** để hiển thị các cuốn sách người dùng đã xem trong thời gian gần đây nhất. Do ta có thêm các nhánh mới cho trang này trong tương lại nên cách thuận tiện nhất là sử dụng **flexbox**. Ta sẽ thêm đoạn CSS này và phân tách phần nội dụng vào trong một thẻ ** ** để sắp xếp theo chiều ngang. Ta sẽ đặt **lớp (class)** của các thẻ ** ** con là **infocell** và chỉnh style với *đường viền xanh và cạnh bo tròn*.

```
<style>
.flexrow {  display: flex;
            border: 2px black;
}
.flexrow > div { flex: 1; }
.infocell {
  border: 2px solid green;
  border-radius: 5px 25px;
  background-color: white;
  padding: 5px;
  margin: 20px 5px 5px 5px;
}
</style>

 <div class="flexrow" >
   <div class="infocell" >
     <p>Profile</p>
     …
   </div>

   <div class="infocell" >
     <p>Viewed Books</p>
     …
   </div>
 </div>
```

9. Chinh sửa thẻ ** ** chứa thẻ ** Viewed Books ** để nếu có cuốn sách thì hiển thị tiêu đề các cuốn sách ở dạng đường link đến trang xem chi tiết từng cuốn sách:

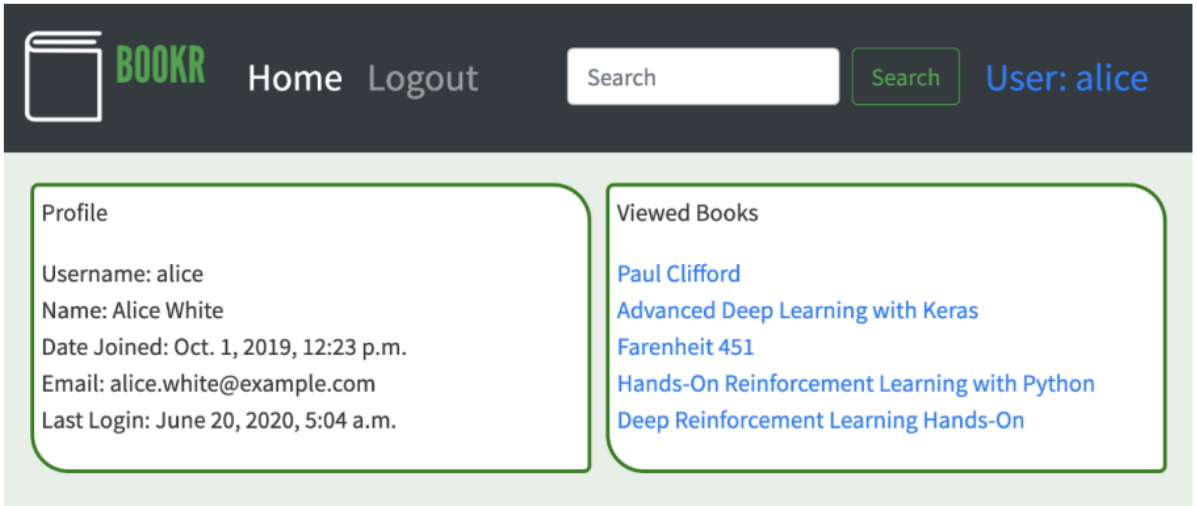
```
<a href="/books/1">Advanced Deep Learning with Keras</a><br>
```

Một thông điệp sẽ xuất hiện nếu danh sách là trống. Toàn bộ thẻ ** ,** bao gồm các vòng lặp quyệt qua **request.session.viewed_books**, sẽ có dạng như sau:

```
    <div class="infocell" >
       <p>Viewed Books</p>
```

```
        <p>
        {% for book_id, book_title in request.session.viewed_books %}
        <a href="/books/{{ book_id }}">{{ book_title }}</a><br>
        {% empty %}
          No recently viewed books found.
        {% endfor %}
        </p>
      </div>
```

10. Sau khi hoàn thành lưu lại các tập tin và mở URL http://127.0.0.1:8000/accounts/profile/, giao diện trang web sẽ như **Hình VI-1**.



Hình VI-1. Danh sách các cuốn sách mà người dùng đã xem gần đây